

Guía para entregar VICAST al cliente

Pasos en el dashboard de Supabase y en tu PC. No va a Play Store: los técnicos usan Expo Go.

20 de septiembre de 2026 · Uso interno (tú), no es el manual del técnico.

Qué ya está en el código y no tienes que programar

Auth, clientes, proyectos, levantamientos, fotos, PDF, perfil, filtros, aviso de borrador, banner de conexión. Si dos técnicos generan el mismo código LEV, la app lo sube al siguiente número solo. No hace falta cola extra ni correlativo SQL para entregar.

No pegues `schema.sql` en un proyecto que ya tiene datos reales.

Ese archivo *borra* las tablas y las vuelve a crear. Úsalo solo si el proyecto está vacío o es de prueba.

0. Qué vas a dejar listo

1. Un proyecto Supabase de producción con tablas, fotos y Auth.
2. La app apuntando a ese proyecto (`.env`).
3. Cuentas de los técnicos (tú las creas; ellos no se registran solos).
4. Backup de base y de fotos.
5. Una forma de abrir la app en el teléfono del cliente (Expo Go).
6. Una prueba completa: login → levantamiento con fotos → PDF.

1. Proyecto Supabase

1. Entra a supabase.com/dashboard.
2. Si el proyecto actual es de pruebas y no te importa perder datos: puedes reutilizarlo. Si ya hay datos de demo que no quieres mezclar con el cliente, crea un proyecto nuevo (New project), región cercana (p. ej. South America), guarda la contraseña de la base.
3. Plan: el gratuito sirve para piloto. Para un cliente con fotos, el plan Pro es más seguro por backups.

2. Aplicar el schema (una sola vez)

Opción A — proyecto vacío (recomendado si es entrega nueva)

1. Dashboard → **SQL Editor** → New query.
2. Abre en Cursor el archivo `supabase/schema.sql`.

3. Copia **todo** el archivo, pégalo, pulsa **Run**.
4. Debe terminar sin error. Si falla a mitad, no lo vuelvas a pegar a ciegas: ese script borra tablas al inicio.

Con esto ya quedan: tablas, RLS, trigger de perfil, bucket privado `fotos`, políticas de Storage, borrar clientes/proyectos, teléfono y foto de perfil. **No corras las migraciones después:** ya están dentro de `schema.sql`.

Opción B — el proyecto ya tiene tablas y datos que quieres conservar

No uses `schema.sql`. En SQL Editor corre, en este orden, solo lo que falte:

1. `supabase/migrations/20260916192500_security_hardening.sql` (si aún no hay bucket `fotos` o columna `activo`).
2. `supabase/migrations/20260918223000_catalog_delete_policies.sql` (si borrar cliente/proyecto falla en la app).
3. `supabase/migrations/20260920180000_profile_phone_photo.sql` (si no existen `telefono` y `foto_path` en perfiles).

Si una política “already exists”, está bien: esa parte ya estaba aplicada.

Comprobar que quedó bien

Table Editor debe mostrar: `perfiles`, `clientes`, `proyectos`, `levantamientos`, `sectores`, `elementos`, `fotos`.

Storage → buckets → `fotos`, privado (no public).

Pega esto en SQL Editor y Run. Todas las filas deben decir `ok`:

```
select 'perfiles' as item, to_regclass('public.perfiles') is not null as ok
union all select 'clientes', to_regclass('public.clientes') is not null
union all select 'proyectos', to_regclass('public.proyectos') is not null
union all select 'levantamientos', to_regclass('public.levantamientos') is not null
union all select 'sectores', to_regclass('public.sectores') is not null
union all select 'elementos', to_regclass('public.elementos') is not null
union all select 'fotos', to_regclass('public.fotos') is not null
union all select 'bucket fotos', exists (select 1 from storage.buckets where id = 'fotos' and public = false)
union all select 'perfiles.telefono', exists (
  select 1 from information_schema.columns
  where table_schema = 'public' and table_name = 'perfiles' and column_name = 'telefono'
)
union all select 'perfiles.foto_path', exists (
  select 1 from information_schema.columns
  where table_schema = 'public' and table_name = 'perfiles' and column_name = 'foto_path'
);
```

3. Auth (para que solo entren los técnicos del cliente)

1. **Authentication** → **Providers** → **Email**: Email habilitado. Confirm email: puedes dejarlo apagado si tú creas las cuentas a mano (más simple para entregar).
2. **Authentication** → **Sign In / Providers** (o Authentication → Settings, según la UI): desactiva **Allow new users to sign up**. Así “Crear cuenta” en la app no sirve para extraños.
3. **Authentication** → **URL Configuration**

- Site URL: la URL de tu proyecto, p. ej. `https://xxxx.supabase.co`
- Redirect URLs, agrega: `vicast://` y `vicast://**` y, si usas web, `http://localhost:8081` y `http://localhost:8081/**`

4. Recuperar clave: la app pide un código de 6 dígitos del correo. En **Authentication** → **Email Templates** → **Reset password** el correo debe incluir `{{ .Token }}` (el código), no solo un enlace.

4. Conectar la app (`.env`)

1. Dashboard → **Project Settings** → **API** (a veces "Data API").
2. Copia **Project URL** y la clave **anon** / **publishable** (la pública, no la `service_role`).
3. En la raíz del proyecto, archivo `.env` (ya está en `.gitignore`, no lo subas a git):

```
EXPO_PUBLIC_SUPABASE_URL=https://TU-PROYECTO.supabase.co
EXPO_PUBLIC_SUPABASE_KEY=eyJ...o la sb_publishable...
```

Reinicia Expo después de cambiar el `.env` (`npx expo start --clear`). Si dejas la clave `service_role`, cualquiera con la app bypassa la seguridad: no la uses.

5. Crear las cuentas de los técnicos

1. Authentication → Users → **Add user** → Create new user.
2. Email real del técnico + contraseña temporal (mínimo 6 caracteres).
3. Marca **Auto Confirm User** para que pueda entrar ya.
4. Al crearlo, el trigger arma una fila en `perfiles`.
5. Table Editor → `perfiles` → edita **nombre** (el que sale en el PDF). Deja `rol = tecnico` y `activo = true`.

Entrégales email + clave temporal y pide que la cambien en la app (olvidé mi contraseña o editar perfil no cambia la clave: usar "¿Olvidaste tu contraseña?").

Si alguien se va: en `perfiles` pon `activo = false`. La app rechaza el login. Opcional: Authentication → el usuario → Ban.

6. Backups

1. Project Settings → Infrastructure / Database → **Backups**.
2. Plan Free: hay respaldo diario limitado. Confirma que aparece al menos un backup.
3. Plan Pro: activa PITR si el cliente lo pide.
4. Storage: las fotos viven en el bucket `fotos`; el backup de Postgres no sustituye del todo las fotos. En Pro, revisa la retención de Storage. No borres el bucket.

7. Cómo se la das al cliente (sin Play Store)

Los teléfonos Android (e iOS) instalan **Expo Go** desde la tienda. Tú arrancas el proyecto y ellos escanean el QR.

1. En tu PC, en la carpeta `revestimientos-industriales` :

```
npm install
npx expo start --clear --tunnel
```

- 2. La primera vez Expo puede pedir cuenta en expo.dev (gratis).
- 3. En el teléfono: abrir Expo Go → escanear el QR.
- 4. `--tunnel` sirve si no están en tu mismo WiFi (visita al cliente). En la oficina, sin tunnel también funciona.
- 5. Cámara y PDF funcionan en el teléfono, no cuentes con el navegador como entrega principal.

Mientras tu PC (o una máquina que dejes encendida) no esté corriendo Expo, ellos no abren la app. Para un piloto de un día está bien. Si necesitan usarla todos los días sin ti, el siguiente paso sería un APK interno con EAS, no Play Store; eso es otro trabajo.

8. Prueba de entrega (hazla tú antes de ir)

- 1. Login con una cuenta de técnico (no la tuya de dashboard).
- 2. Editar perfil: nombre, teléfono, foto.
- 3. Alta corta cliente + proyecto + tipo de servicio.
- 4. Wizard: condiciones, un sector, 2 fotos (cámara y galería).
- 5. Cerrar levantamiento → compartir PDF.
- 6. Modo avión: editar un borrador, quitar avión, ver que el banner sincroniza y las fotos aparecen en Storage → `fotos` .
- 7. Cerrar sesión y volver a entrar: deben seguir los datos.
- 8. En Table Editor, el levantamiento debe existir con su código `LEV-2026-...` .

9. Qué le dices al cliente el día 1

TEMA	MENSAJE
Cuentas	Cada técnico tiene su email. No se registran solos. No se ven los levantamientos de otro.
Obligatorio para empezar	Nombre de cliente, nombre de recinto, tipo de servicio. El RUT y datos del PDF se pueden completar después (sale un aviso).
Sin señal	Pueden trabajar. Al volver internet, se sube solo. Dejar la app abierta un momento al reconectar.
Informe	Se genera el PDF en el teléfono y se comparte (WhatsApp, correo, Drive).
Soporte	Tú creas/desactivas usuarios en Supabase. Ellos no administran el backend.

10. Checklist final

ÍTEM	
☐	SQL aplicado y query de comprobación todo <code>ok</code>
☐	Bucket <code>fotos</code> privado
☐	Sign up público desactivado
☐	<code>.env</code> con URL y clave anon del proyecto de entrega
☐	Un usuario por técnico, nombre bien en <code>perfiles</code>
☐	Backup visible en el dashboard
☐	Prueba completa en teléfono (fotos + PDF + offline)
☐	Expo Go + QR (o tunnel) probado en un teléfono que no sea el tuyo

Archivos de esta guía: `docs/guia-entrega-cliente.html` y `docs/guia-entrega-cliente.pdf`.